

NTNU

Norwegian University of
Science and Technology

DEPARTMENT OF COMPUTER SCIENCE

TDT4200 - PARALLEL PROGRAMMING

Exercise 7

1 Introduction

In this assignment, we will parallelise the 2D wave equation using an accelerator with CUDA. GPU parallelisation offers the same advantage as thread parallelisation: we can parallelise the code incrementally, rather than all at once, as with MPI.

The 2D wave equation is given by

$$\frac{\partial^2 u}{\partial t^2} = c^2 \cdot \left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} \right), \tag{1}$$

where

- u is the wave's amplitude
- t is time
- x and y are spatial
- c is the wave's propagation rate (celerity)

visualised in Figure 1.

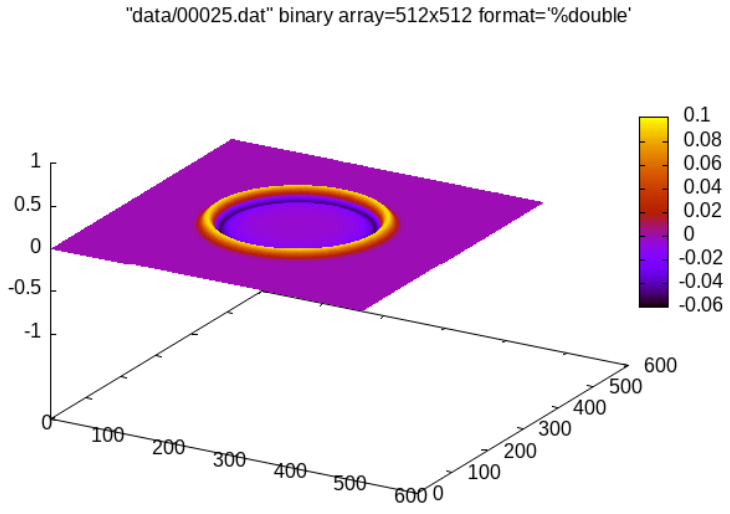


Figure 1: Visualisation of the 2D wave equation.

Recall the central finite difference approximation

$$f''(x) \approx \frac{f(x-h) - 2 \cdot f(x) + f(x+h)}{h^2}.$$

Equation 1 can be discretised in space with the central Finite Differences Method as

$$f''(x,y) \approx \frac{u^t_{i-1,j} - 2u^t_{i,j} + u^t_{i+1,j}}{\Delta x^2} + \frac{u^t_{i,j-1} - 2u^t_{i,j} + u^t_{i,j+1}}{\Delta y^2},$$

and in time

$$f''(t) \approx \frac{f(t-\Delta t) + f(t+\Delta t) - 2 \cdot f(t)}{\Delta t^2},$$

yielding

$$\frac{u_i^{t-1} - 2 \cdot u_i^t + u_i^{t+1}}{\Delta t^2} = c^2 \cdot \left(\frac{u_{i-1,j}^t - 2u_{i,j}^t + u_{i+1,j}^t}{\Delta x^2} + \frac{u_{i,j-1}^t - 2u_{i,j}^t + u_{i,j+1}^t}{\Delta y^2} \right) \tag{2}$$

combined.

Setting $h = \Delta x = \Delta y$, and re-ordering for the point we're interested in yields

$$u_{ij}^{t+1} = -u_{ij}^{t-1} + 2u_{ij}^t + \frac{\Delta t^2 c^2}{h^2} \cdot (u_{i-1,j}^t + u_{i+1,j}^t + u_{i,j-1}^t + u_{i,j+1}^t - 4u_{i,j}^t) \tag{3}$$

.

Figure 2 is a visualisation of Equation 3

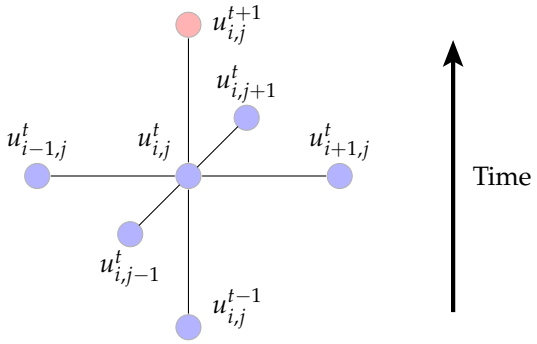


Figure 2: To calculate the red cell, $u_{i,j}^{t+1}$, we are using five known points in space and two known points in time.

2 Tasks

2.1 Programming

Note that although the handout code looks like C, CUDA is a programming model extending C++. In this assignment, we will utilise `Coorperative Groups`, a powerful abstraction in modern CUDA programming that lets the programmer organise, synchronise, and coordinate subsets of threads.

CUDA threads are organised as illustrated in Figure 3.

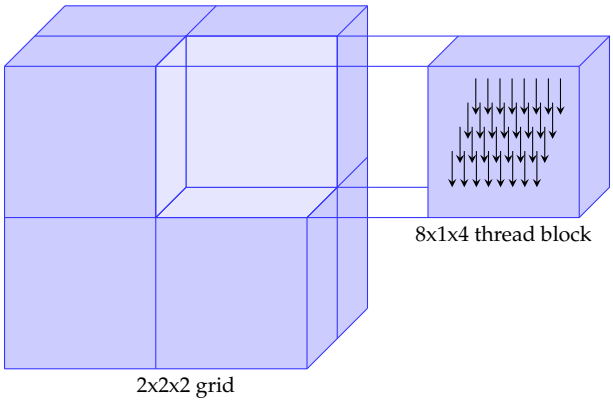


Figure 3: Threads are contained within **blocks** that are organised in **grids**.

For this problem set, we have set the default domain size to be small enough to fit on the GPUs ($128x128$), but feel free to play around with M and N to see how it performs.

Before you start the exercise, it is recommended to run the `make check` command from the `Makefile`, which will compare the sequential and parallel solutions. Running it on the handout files should produce a similar result as shown in Figure 4.

```
find: 'data': No such file or directory
find: 'data': No such file or directory
mkdir -p data images
gcc wave_2d_sequential.c -std=c99 -O2 -Wall -Wextra -o sequential -lm
nvcc wave_2d_parallel.cu -O2 -fmad=false -o parallel -lm
mkdir -p data_sequential
./sequential
Total elapsed time: 15.789879 seconds
cp -rf ./data/* ./data_sequential
./parallel
Total elapsed time: 15.673843 seconds
python3 compare.py data_sequential/00000.dat data/00000.dat

Data files data_sequential/00000.dat and data/00000.dat are identical within the margin of 0.0001

python3 compare.py data_sequential/00075.dat data/00075.dat

Data files data_sequential/00075.dat and data/00075.dat are identical within the margin of 0.0001

rm -rf data_sequential
```

Figure 4: Running `make check` on a fresh project. The tests succeeded because we haven't implemented parallelisation yet; the parallel file is actually a sequential version.

In addition to the `make check` command, the script `plot_differences` will help you with debugging. The script takes two arguments, paths to two output files for comparison.

```
python3 plot_differences.py <path_to_sequential_output> <path_to_parallel_output>
```

Listing 1: How to run the script.

The following command is an example of a run

```
python3 plot_differences.py data_sequential/00075.dat data/00075.dat
```

Listing 2: Example of comparing state 75,

This script stores a `.png` with the differences between the two output files as shown in Fig. 5.

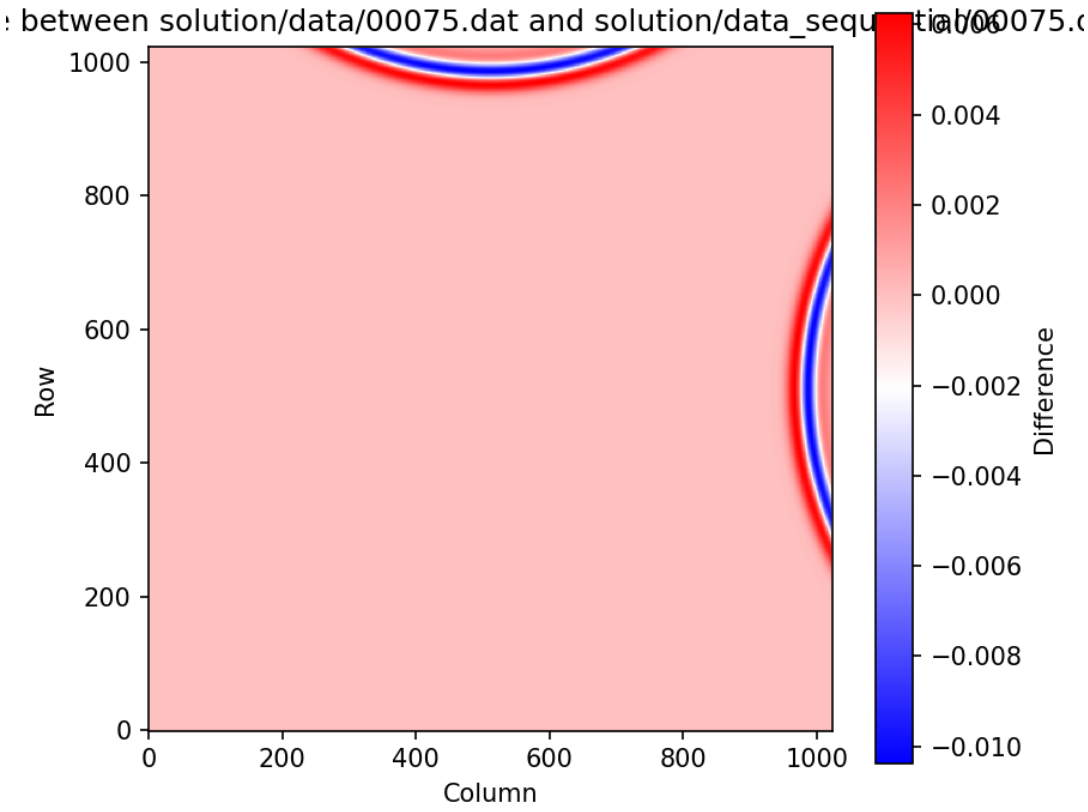


Figure 5: Comparing the output from the sequential and parallel versions. It looks like something is wrong with two of the boundaries.

1. **Include Cooperative Groups:**

   To use cooperative groups, we need to include the library and access it through the `cooperative_groups` namespace.

   a Include the cooperative group header.

   b Declare variables that you find fitting for this exercise.

   Hint: It is helpful for readability to prefix variables on the host with `h_` and variables on the device with `d_`.

2. **Initialise CUDA:**

   We need to know that we have a CUDA-compatible GPU. Using the `cudaGetDeviceCount` function returns the number of available devices. When we have decided which device to use, we can set it with `cudaSetDevice`. We can now print some information about the device by printing attributes of the struct `cudaDeviceProp` and using the function `cudaGetDeviceProperties`.

Implement the function `init_cuda`. The function should:

- Check whether a CUDA-compatible device is connected. If not, return `false`.
- Use a compatible device.
- Use `cudaDeviceProp` to print information similar to what's shown in Figure 6.
- If any errors occur while accessing the properties, return `false`; otherwise, return `true`.

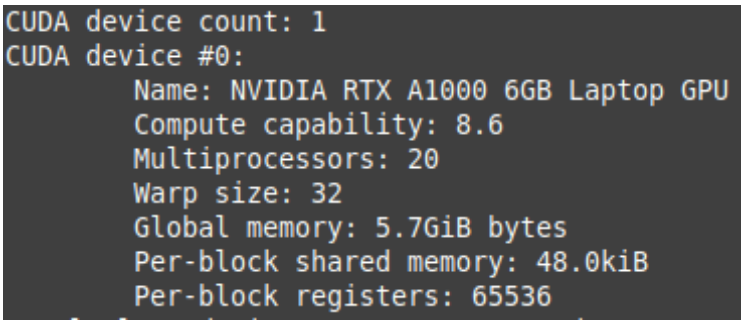


Figure 6: After implementing T2, running the parallel version should print something similar.

3. **Initialise the Domain:**

   Before the GPU can do any calculations, the data needs to be transferred to the device memory.

   Change the function `domain_initialize`, such that
   - It allocates space for at least one grid on the host, which can be written to a file.
   - It allocates space for three grids on the device (prv, current, and nxt).
   - It transfers suitable variables to the device for calculations.

4. **Finalise the Domain:**
   After allocating memory, we must also return it to the OS to prevent memory leaks.

   Change the function `domain_finalize` such that it frees all the memory allocated by `domain_initialize`.

5. **Time Step:**

   The central part we want to parallelise is the calculation of the space domain.

   Change the function `time_step` such that it can be launched as a kernel on the GPU from the CPU.

   Note: Depending on how you design your solution, you might want to change the `move_buffer_window` function as well.

   Hint: CUDA kernel launches are asynchronous. While debugging, use `cudaGetLastError()` after a kernel launch to detect launch errors. A subsequent `cudaDeviceSynchronize()` can be

used to expose errors that occur during kernel execution. Avoid unnecessary synchronisation when measuring performance.

6. **The Boundary Condition:**

   The boundary condition is embarrassingly parallel and well-suited for GPU computation.

   > Change the function `boundary_condition` to calculate the ghost cells on the GPU.

   Hint: Does this kernel need to be launched from the CPU? Or can it be called from the `time_step` function?

7. **Combining it all Together:**

   Now, we must combine the updated simulation and launch the kernel from the CPU.

   > Change the `simulate` function such that it prepares the grid and launches the kernel using cooperative groups. Remember to copy the answer to the CPU for data storage whenever the time iteration matches the snapshot frequency.

8. **Performance - Occupancy:**

   GPU occupancy is the ratio between the number of active warps on a streaming multiprocessor (SM) and the maximum number of warps that the SM can support. The theoretical occupancy, the utilisation of the GPU, can be described as

$$\text{occupancy} = \frac{\text{active warps}}{\text{maximum warps per multiprocessor}}, \tag{4}$$

   where active warps is given by

$$\text{active warps} = \text{active blocks pr SM} \times \text{warps per block}.$$

   > Implement the function `occupancy` such that it estimates the theoretical occupancy of your `time_step` kernel for the block size used in your program (Eq.(4).

   Hint: `cudaGetDeviceProperties` contains information about the GPU. `cudaOccupancyMaxActiveBlocksPerMultiprocessor` can determine how many blocks of your kernel can be active simultaneously on one SM.

```
Grid size set to:              (5, 5)
Launched blocks of size:       (27, 28)
Theoretical occupancy:         0.979167
```

Figure 7: The `occupancy` function should print something similar to this after implementation.

2.2 Theory Questions

1. What is the difference between a function declared as `__global__` and a function declared as `__device__` in CUDA?

   Solution: A function declared as `__global__` can only be called from the CPU while executing the function on the GPU. On the other hand, a function declared as `__device__` can only be called from the GPU. This also executes on the GPU.

2. What are some advantages of parallelising with CUDA compared to MPI?

3. What are some pros and limitations when using cooperative groups?

4. What occupancy did you achieve on the GPU? Can you improve it?

3 Deliverables

Please submit a `.zip` file with the following contents containing your changes to the code:

- `wave_2d_parallel.cu`
- A document with your answers to the theory questions.